

Smart Supply Chain

Codebase Learning Guide

Angular 21 · FastAPI · MongoDB · Prophet · KNN · Ollama

What this guide covers

■ Auth Service & JWT Flow	BehaviorSubject, RxJS operators, token lifecycle
■ Routing & Guards	CanActivateFn, role-based redirects, undefined filtering
■ Sales Order Component	Parallel HTTP, computed getters, scatter plot, calendar
■ Demand Forecast Component	loadId pattern, stddev thresholds, polling
■ FastAPI Setup & CORS	Lifespan hooks, middleware, router prefixes
■ JWT Authentication	Token creation, Depends(), password hashing
■ Pydantic Models	Base → Create → Out pattern, automatic field filtering
■ MongoDB & Motor	Async queries, singleton pattern, _id conversion
■ KPI Router	BackgroundTasks, enriched order data, anomaly status
■ KNN Anomaly Detection	Feature vectors,.pkl model, fraud classification
■ Prophet Forecasting	log1p, asyncio.to_thread, 90-day horizon
■ Local LLM via Ollama	Model selection, graceful fallback, prompt engineering
■ HTTP + Token Full Flow	End-to-end request lifecycle, expiry handling
■ Rebuild Checklist	4-phase step-by-step rebuild guide

Project Overview & File Structure

A Smart Supply Chain Dashboard — a full-stack web app for tracking orders, detecting fraud, forecasting demand, and generating AI summaries using a local LLM. Here is every file and what it actually does.

Tech Stack

Layer	Technology	Role in this project
Frontend	Angular 21 (standalone)	SPA, components, services, reactive forms
Backend	FastAPI (Python)	Async REST API, JWT auth, CORS
Database	MongoDB + Motor	Stores orders, products, forecasts, users
Auth	JWT (python-jose)	Signed token issued on login, sent on every API call
ML	scikit-learn (KNN)	Detects unusual/fraud orders by feature similarity
Forecasting	Prophet (Meta)	Time-series demand prediction, retrains in background
AI Text	Ollama (local LLM)	Generates natural language summaries — qwen2.5/llama3

Full File Structure

Frontend Structure

```
-- FRONTEND (Angular 21) --
src/
  services/
    auth.ts      <- Auth service: login, logout, token, userState$
  app/
    app.config.ts  <- provideRouter + provideHttpClient
    app.routes.ts  <- All routes and which guard protects them
    auth.guard.ts  <- Blocks unauthenticated users, redirects to /login
    login.guard.ts <- Redirects already-logged-in users away from /login
    login/         <- Login form component (email, password, signals)
    sales-order/   <- Orders dashboard (most complex component)
    demand-forecast/ <- Forecast calendar + chart component
    admin-dashboard/ <- Admin panel (placeholder, empty component)
    chatbot/       <- Chatbot (placeholder)
    supplier/      <- Supplier view (placeholder)
```

Backend Structure

```
-- BACKEND (FastAPI) --
app/
  main.py          <- Entry: CORS middleware, lifespan, router registration
  core/
    config.py      <- Settings via pydantic-settings: JWT secret, MongoDB URI
    database.py    <- MongoDB connection singleton + get_db() dependency
    security.py    <- Password hashing (pbkdf2) + JWT token creation
  routers/
    auth.py        <- POST /api/auth/login, GET /api/auth/me
    users.py       <- CRUD for admin users (list, create, get, update, delete)
    kpis.py        <- Orders, purchases, products, forecasts, AI explain
  services/
    auth_service.py <- get_current_admin() dependency (token -> user lookup)
    forecast_service.py <- Prophet retrain + LLM explanation generation
  models/
    user.py        <- Pydantic schemas: AdminBase/Create/Out, LoginRequest
    kpi.py         <- Schemas: AnomalyRecord, RFMRecord, DemandForecast
    order.py       <- Schema: SalesOrder with order_lines[]
    product.py     <- Schema: Product and Department
```

Angular Concepts Used in This Project

You already know two-way binding, services, and routing. Here are the more advanced patterns your project uses and exactly why.

BehaviorSubject — The Auth State Container

A BehaviorSubject is a special kind of Observable that always holds the latest value and immediately gives it to new subscribers. It is the core of your auth system.

`auth.ts` — BehaviorSubject

```
// In auth.ts – three possible states:
//   undefined -> app just loaded, still checking token
//   null       -> checked, no valid session
//   UserState  -> checked, user is logged in

private userStateSubject = new BehaviorSubject<UserState | null | undefined>(undefined);
public userState$ = this.userStateSubject.asObservable();

// Updating state – pushes new value to all subscribers:
this.userStateSubject.next(null); // logged out
this.userStateSubject.next(user); // logged in
```

Why undefined as the initial value? Because null means 'logged out' and undefined means 'don't know yet'. Guards filter out undefined so they never accidentally redirect while the auth check is still in progress.

RxJS Operators Used

Operator	What it does	Where used in your code
<code>tap(fn)</code>	Run side effect without changing the stream	Save token to localStorage after login
<code>switchMap(fn)</code>	Replace stream with a new Observable	After login: switch to <code>fetchCurrentUser()</code> call
<code>catchError(fn)</code>	Handle errors, return fallback Observable	If <code>/auth/me</code> fails: logout + return of(<code>null</code>)
<code>of(value)</code>	Create Observable that emits one value immediately	Return null when no token exists
<code>map(fn)</code>	Transform each emitted value	Convert user object to true/false in <code>checkAuth()</code>
<code>filter(fn)</code>	Only pass values that match condition	Guards: skip undefined, wait for real auth state
<code>take(1)</code>	Complete after first emission	Guards only need to check once, not keep watching
<code>pipe(...ops)</code>	Chain multiple operators together	Used on every HTTP call in <code>auth.ts</code>

inject() — Modern Angular DI

Your project uses the newer `inject()` function instead of constructor injection. Both do the same thing — `inject()` just works without a constructor, which is cleaner for standalone components and guard functions.

Dependency Injection styles

```
// Old style (still valid):
constructor(private auth: Auth, private router: Router) {}

// New style (used in your project):
private auth = inject(Auth);
private router = inject(Router);
// No constructor needed at all
```

Signals — New State Primitives

The Login component uses Angular Signals for `errorMessage` and `isLoading`. Signals are simpler than `BehaviorSubject` for local component state — no `subscribe()` needed.

login.ts — Signals

```
// Declare a signal with initial value:
errorMessage = signal<string | null>(null);
isLoading     = signal<boolean>(false);

// Update it:
this.errorMessage.set('Please fill out all fields.');
```

```
this.isLoading.set(true);

// Read it in template (no async pipe needed):
// {{ errorMessage() }}    <- Note: called as a function
```

Standalone Components (Angular 21)

Your project uses standalone components — the modern Angular approach. Each component declares its own imports instead of a shared `NgModule` file.

Standalone component pattern

```
@Component({
  selector: 'app-sales-order',
  standalone: true,           // <- no NgModule file needed
  imports: [CommonModule,    // <- *ngIf, *ngFor, pipes
            FormsModule],    // <- [(ngModel)] two-way binding
  templateUrl: './sales-order.html',
  styleUrls: ['./sales-order.css'],
})
export class SalesOrder implements OnInit {
  ngOnInit(): void {
    this.loadDashboardData(); // called once when component mounts
  }
}
```

Computed Getters — Live Derived State

The sales order component uses TypeScript getters for filtered and paginated orders. Angular re-evaluates these on every change detection cycle, so the template always shows up-to-date data.

`sales-order.ts — Getters`

```
// Getter = computed property, no () in template
get filteredOrders(): any[] {
  const q = this.tableSearchQuery.toLowerCase();
  if (!q) return this.orders;
  return this.orders.filter(o =>
    o.id.toString().includes(q) || o.status.toLowerCase().includes(q)
  );
}

get paginatedOrders(): any[] {
  const start = (this.currentPage - 1) * this.pageSize;
  return this.filteredOrders.slice(start, start + this.pageSize);
}

// In template: *ngFor="let order of paginatedOrders"
// NOT: paginatedOrders() <- no parentheses needed
```

Auth Service — Full Breakdown

The Auth service is the single source of truth for authentication state in the entire app. Every guard, component, and HTTP call that needs to know "am I logged in?" goes through this service.

Login Flow — Step by Step

auth.ts — login()

```
login(email: string, password: string): Observable<UserState | null> {  
  return this.http.post<{ access_token: string }>(  
    `${this.apiUrl}/auth/login`,  
    { email, password }           // Step 1: POST credentials  
  ).pipe(  
    tap(response => {  
      localStorage.setItem('access_token', response.access_token);  
    }),                          // Step 2: Save token (side effect)  
    switchMap(() => this.fetchCurrentUser()) // Step 3: Who am I?  
  );  
}
```

After login: the token is in localStorage, and userState\$ emits the full user object. Any component or guard subscribed to userState\$ immediately sees the change.

Session Restore on App Load

When the Angular app starts, the Auth service constructor runs immediately. It checks if there is a saved token and validates it against the backend — so refreshing the page does not log you out.

auth.ts — checkAuth() + fetchCurrentUser()

```
constructor() {
  this.checkAuth().subscribe(); // always runs on startup
}

fetchCurrentUser(): Observable<UserState | null> {
  const token = localStorage.getItem('access_token');
  if (!token) {
    this.userStateSubject.next(null); // no token = logged out
    return of(null);
  }
  const headers = new HttpHeaders({ 'Authorization': `Bearer ${token}` });
  return this.http.get<UserState>(`${this.apiUrl}/auth/me`, { headers }).pipe(
    tap(user => this.userStateSubject.next(user)), // update state
    catchError(() => {
      this.logout(); // bad token -> clean up
      return of(null);
    })
  );
}
```

Route Guards

auth.guard.ts + login.guard.ts

```
// authGuard — blocks unauthenticated access
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(Auth);
  const router = inject(Router);
  return auth.userState$.pipe(
    filter(user => user !== undefined), // wait for auth check to finish
    take(1), // only check once
    map(user => {
      if (user) return true; // logged in -> allow
      router.navigate(['/login']); // not logged in -> redirect
      return false;
    })
  );
};

// loginGuard — redirects already-logged-in users + role routing
map(user => {
  if (user) {
    if (user.role === 'supplier') router.navigate(['/supplier']);
    else router.navigate(['/']);
    return false; // don't show the login page
  }
  return true; // not logged in -> show login
})
```


Key insight: the filter(`user !== undefined`) is what prevents a race condition. Without it, a guard could see undefined (still loading) and assume nobody is logged in, causing an unwanted redirect to `/login`.

Sales Order Component

The most complex component in the project. It fetches orders, enriches them with computed values, builds a scatter plot (pure SVG via coordinate math), renders a monthly calendar, and re-implements a mini KNN in TypeScript for visualisation.

Parallel API Calls

Three independent HTTP calls fire simultaneously in `ngOnInit`. They do not wait for each other — each updates its own part of the UI when it arrives.

`sales-order.ts` — `loadDashboardData()`

```
loadDashboardData(): void {
  const headers = new HttpHeaders({ 'Authorization': `Bearer ${token}` });

  // Call 1 — orders: triggers scatter plot and calendar build
  this.http.get<any[]>('http://127.0.0.1:8000/api/kpis/orders', { headers })
    .subscribe({
      next: (data) => {
        this.orders = data;
        this.buildScatterPlot();
        this.buildCalendarGrid();
      },
      error: (err) => console.error(err)
    });

  // Call 2 — purchases (fires independently, same time)
  this.http.get<any[]>('http://127.0.0.1:8000/api/kpis/purchases', { headers })
    .subscribe({ next: (data) => { this.purchases = data; } });

  // Call 3 — AI executive summary (may take a few seconds)
  this.http.get<any>('http://127.0.0.1:8000/api/kpis/overview/explain', { headers })
    .subscribe({ next: (res) => { this.aiExplanation = res.explanation; } });
}
```

Client-side KNN Nearest Neighbour Explainer

When you click a scatter point, the frontend computes which 5 orders are most similar — using Euclidean distance across 4 normalised features. This is NOT the backend model; it is a TypeScript reimplementation for visualisation only.

sales-order.ts — computeNearestNeighbors()

```
computeNearestNeighbors(selectedPt: ScatterPoint): void {
  const normFn = (val, min, max) => (val - min) / (max - min || 1);

  const normSelected = {
    sales: normFn(selectedOrder.total_sales, this.minSales, this.maxSales),
    margin: normFn(selectedOrder.profit_margin, this.minProfit, this.maxProfit),
    qty: normFn(selectedOrder.total_quantity, this.minQty, this.maxQty),
    delay: normFn(selectedOrder.delay_delta, this.minDelay, this.maxDelay),
  };

  const distances = this.orders
    .filter(o => o.id !== selectedPt.id)
    .map(o => {
      const normO = { /* same normFn for each feature */ };
      const dist = Math.sqrt(
        (normSelected.sales - normO.sales) ** 2 +
        (normSelected.margin - normO.margin) ** 2 +
        (normSelected.qty - normO.qty) ** 2 +
        (normSelected.delay - normO.delay) ** 2
      );
      return { order: o, dist };
    });

  distances.sort((a, b) => a.dist - b.dist);
  this.nearestNeighbors = distances.slice(0, 5);
}
```

Calendar Grid Builder

The calendar is built entirely in TypeScript — no library. It calculates which day of the week each month starts on, fills in trailing/leading days from adjacent months, then maps order events to date strings.

sales-order.ts — buildCalendarGrid()

```
buildCalendarGrid(): void {
  const firstDayIndex = new Date(year, month, 1).getDay(); // 0=Sun
  const numDays = new Date(year, month + 1, 0).getDate(); // last day of month

  // Fill trailing days from previous month
  for (let i = firstDayIndex - 1; i >= 0; i--) { ... }
  // Fill current month days + attach events
  for (let i = 1; i <= numDays; i++) {
    const dateStr = `${year}-${pad(month+1)}-${pad(i)}`;
    cells.push({ dayNumber: i, isCurrentMonth: true,
      events: getEventsForDate(dateStr) });
  }
  // Fill leading days to reach 35 or 42 cells total
}
```

Demand Forecast Component

This component handles product selection, fetches Prophet forecasts, builds the chart, and populates the calendar with predicted stockout events. It has the most complex async patterns in the project.

The Load ID Anti-Stale Pattern

If the user switches products quickly, responses from older requests can arrive after newer ones. The load ID pattern detects and ignores stale responses.

`demand-forecast.ts` — `loadId` pattern

```
private _currentLoadId = 0;

loadEventsAndBuildCalendar(): void {
  const loadId = ++this._currentLoadId; // new ID for this call

  // Safety timer: stop spinner after 5s even if API is slow
  this._safetyTimer = setTimeout(() => {
    if (loadId !== this._currentLoadId) return; // stale? ignore
    this.isLoadingCalendar = false;
    this.buildCalendarEvents([], []);
  }, 5000);

  this.http.get(`/api/kpis/forecasts?product_id=${this.selectedProductId}`)
    .pipe(timeout(15000))
    .subscribe({
      next: (forecasts) => {
        if (loadId !== this._currentLoadId) return; // user switched product
        clearTimeout(this._safetyTimer);
        this.buildCalendarEvents(forecasts);
        this.isLoadingCalendar = false;
      }
    });
}
```

Without the `loadId` check: click Product A, then quickly click Product B. If Product A's slower response arrives after Product B's, it overwrites the screen with Product A's data. The `loadId` detects this and discards the stale response.

Dynamic Thresholds using Standard Deviation

Instead of a fixed number to define "high demand", the component calculates it dynamically each month based on that product and month's distribution.

demand-forecast.ts — dynamic thresholds

```
const values = monthForecasts.map(f => f.forecast);
const mean   = values.reduce((s, v) => s + v, 0) / values.length;
const stdDev = Math.sqrt(
  values.reduce((s, v) => s + (v - mean) ** 2, 0) / values.length
);

// 1.2 std deviations above average = high demand day
const highDemandThreshold = mean + 1.2 * stdDev;
// 1.7 std deviations above average = potential stockout
const stockoutThreshold   = mean + 1.7 * stdDev;
```

Background Polling for Prophet Results

After receiving forecasts, the component checks whether December data exists. If Prophet has not finished training yet, it re-polls every 2 seconds (up to 5 times).

demand-forecast.ts — polling

```
const hasDecemberForecasts = forecasts.some(
  f => f.sales === null && f.date >= '2017-12-01'
);

if (!hasDecemberForecasts && this.pollCount < 5) {
  this.pollCount++;
  // Try again in 2 seconds
  this._pollingTimer = setTimeout(() => {
    this.loadEventsAndBuildCalendar();
  }, 2000);
}
```

FastAPI Setup, CORS & Routing

How the backend boots, connects to MongoDB on startup, and registers all routes under the /api prefix.

Lifespan — Startup and Shutdown

main.py — lifespan

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await db_connection.connect_to_database() # runs on startup
    yield # app runs here
    await db_connection.close_database_connection() # on shutdown

app = FastAPI(title="Smart Supply Chain Backend", lifespan=lifespan)
```

The yield is the key. Code before yield = startup. Code after yield = shutdown. This is the modern FastAPI way to manage resources that need to open and close around the app's lifetime.

CORS Middleware

main.py — CORS

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:4200", # Angular dev server
        "http://127.0.0.1:4200",
    ],
    allow_credentials=True, # required for Authorization headers
    allow_methods=["*"], # GET, POST, PUT, DELETE, OPTIONS, etc.
    allow_headers=["*"], # including Authorization
)
```

REBUILD WARNING: If you deploy Angular on a different domain (e.g. Vercel), add that URL to allow_origins. Without it, every API call will fail silently in the browser with a CORS error.

Router Registration & Prefix Chaining

main.py — router registration

```
# Each router file has its own inner prefix:
# auth.router    has prefix="/auth"    -> full path: /api/auth/login
# users.router   has prefix="/admins"  -> full path: /api/admins/
# kpis.router    has prefix="/kpis"    -> full path: /api/kpis/orders

app.include_router(auth.router, prefix="/api")
app.include_router(users.router, prefix="/api")
app.include_router(kpis.router, prefix="/api")
```

Depends() — FastAPI Dependency Injection

`Depends()` is FastAPI's way of sharing reusable logic across routes. When you add `Depends(get_db)` or `Depends(get_current_admin)` to a route parameter, FastAPI runs that function first and injects the result.

How `Depends()` works

```
# Dependency: provides a DB instance to any route that needs it
async def get_db():
    return db_connection.db

# Usage in a route: FastAPI calls get_db() automatically
@router.get("/orders")
async def list_orders(db = Depends(get_db), current_admin = Depends(get_current_admin)):
    # db is already connected, current_admin is already validated
    cursor = db["sales_orders"].find().limit(500)
    ...

# If get_current_admin raises HTTPException(401),
# the route function never runs at all.
```

JWT Authentication — Full Flow

How a username + password becomes a signed token, and how that token proves identity on every subsequent request without touching the database for every check.

What a JWT Contains

JWT structure

```
# A JWT is three Base64-encoded parts joined by dots:
# header.payload.signature

# Decoded payload for your project:
{
  "sub": "60c72b2f9b1d8e2e4c8d9e01", # admin's MongoDB _id
  "exp": 1719500000                # expiry (Unix timestamp)
}

# The signature is created with:
# HMACSHA256(header + "." + payload, JWT_SECRET_KEY)
# Only the server knows JWT_SECRET_KEY, so nobody can fake a token.
```

Token Creation — security.py

core/security.py

```
def create_access_token(subject: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=120)
    to_encode = {"exp": expire, "sub": str(subject)}
    return jwt.encode(to_encode, settings.JWT_SECRET_KEY,
                      algorithm="HS256")

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password) # pbkdf2_sha256

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)
```

Login Route — routers/auth.py

outers/auth.py

```
@router.post("/login")
async def login(login_data: LoginRequest, db = Depends(get_db)):
    email = login_data.email.lower()
    admin = await db["admin"].find_one({"email": email})

    if not admin or not verify_password(login_data.password,
                                         admin["hashed_password"]):
        raise HTTPException(status_code=400,
                            detail="Incorrect email or password")

    access_token = create_access_token(subject=str(admin["_id"]))
    return {"access_token": access_token, "token_type": "bearer"}
```

Token Validation — get_current_admin()

services/auth_service.py

```
async def get_current_admin(
    credentials: HTTPAuthorizationCredentials = Depends(HTTPBearer()),
    db = Depends(get_db)
) -> dict:
    token = credentials.credentials # "Bearer <token>" -> just token

    # Decode + verify the JWT signature
    payload = jwt.decode(token, settings.JWT_SECRET_KEY,
                          algorithms=["HS256"])
    admin_id = payload.get("sub") # MongoDB _id string

    # Look up the admin in MongoDB
    admin = await db["admin"].find_one({"_id": ObjectId(admin_id)})
    if admin is None:
        raise HTTPException(status_code=401)

    admin["id"] = str(admin["_id"])
    return admin # injected into every protected route
```

Pydantic Models & MongoDB

The Base → Create → Out Pattern

Every entity in your project follows this three-class pattern. It controls exactly which fields appear in requests vs responses — and automatically hides sensitive data like hashed passwords.

models/user.py — Admin pattern

```
class AdminBase(BaseModel):
    email: EmailStr          # validated email format
    first_name: str
    last_name: str
    role: str = "admin"      # default value

class AdminCreate(AdminBase):
    password: str            # only needed when creating (POST body)

class AdminOut(AdminBase):
    id: str                  # only sent in responses, never receives
    # NOTE: hashed_password is NOT here – Pydantic strips it from all responses
    class Config:
        populate_by_name = True
```

Security by design: using AdminOut as response_model in the route decorator means FastAPI filters the MongoDB document through AdminOut before returning it. Even if the document has hashed_password, it never leaves the server.

Demand Forecast Model

models/kpi.py — DemandForecast

```
class DemandForecastBase(BaseModel):
    date: str                # "YYYY-MM-DD"
    product_id: Optional[int] = None
    sales: Optional[float] = None # None = future date (no actual sale)
    forecast: float           # Prophet's predicted value

# Angular uses sales == null to decide:
#   sales != null -> draw solid "actual" line on chart
#   sales == null -> draw dashed "forecast" line on chart
```

MongoDB + Motor — Async Queries

Motor async query patterns

```
# Find one document
admin = await db["admin"].find_one({"email": email})

# Find many — Motor returns an async cursor
records = []
async for doc in db["forecasts"].find(query).sort("date", 1).limit(2000):
    records.append(doc)

# Insert and capture the new _id
result = await db["admin"].insert_one(admin_dict)
new_id = str(result.inserted_id)

# Update one document
await db["admin"].update_one({"_id": ObjectId(id)}, {"$set": updates})

# Delete
await db["admin"].delete_one({"_id": ObjectId(id)})

# ALWAYS convert _id to string for JSON serialization:
doc["id"] = str(doc.pop("_id")) # pop removes _id, sets id
```

Motor is the async version of PyMongo. Every query uses `await`, so FastAPI can handle other requests while MongoDB is responding — no blocking.

KPI Router — Orders, Forecasts & AI

The largest router in the project. It enriches raw database records with computed fields, runs ML predictions, triggers background tasks, and calls the local LLM.

Background Tasks for Non-Blocking Retraining

`routers/kpis.py` — BackgroundTasks

```
@router.get("/forecasts")
async def list_forecasts(
    product_id: int,
    background_tasks: BackgroundTasks, # injected by FastAPI
    db = Depends(get_db),
    current_admin = Depends(get_current_admin)
):
    # Check if December forecasts already exist
    has_future = await db["forecasts"].find_one({
        "product_id": product_id,
        "sales": None,
        "date": {"$gte": "2017-12-01"}
    })

    if not has_future:
        # Kick off Prophet retraining WITHOUT blocking the HTTP response
        background_tasks.add_task(retrain_demand_forecast, db, product_id)

    # Return current data immediately (Angular polls for updated results)
    cursor = db["forecasts"].find(query).sort("date", 1).limit(2000)
    return [format_mongo_doc(doc) async for doc in cursor]
```

What `/api/kpis/orders` Computes

The orders route does not just return raw documents. For each order it joins with the products collection, computes aggregate values, and runs the KNN model:

Computed Field	How it is calculated
category	Joins <code>order_lines[0].product_sku</code> with products collection
total_quantity	Sum of <code>order_lines[].quantity</code>
total_sales	Sum of <code>quantity * unitPrice</code> for each line
profit_margin	<code>order_profit / total_sales</code>
discount_ratio	Average discount across all order line products
delay_delta	<code>real_shipment - scheduled_shipment</code> (negative = early)
anomaly_status	"unusual" "delay anomaly" "valid" based on KNN + <code>delay_delta</code>

KNN Anomaly Detection

K-Nearest Neighbors classifies each sales order as normal (valid), suspicious (unusual), or delayed. It works by comparing a new order to its most similar historical orders.

How KNN Works in Plain English

Imagine plotting every order on a graph with axes: sales value, profit margin, delivery delay, quantity. Fraudulent orders tend to cluster in unusual regions. KNN looks at an order's 5 closest neighbours and asks "are most of them fraud cases?" If yes, this order is probably fraud too.

Feature Vector & Prediction

routers/kpis.py — KNN prediction

```
# 5 features sent to the trained KNN model for each order
features_df = pd.DataFrame([[
    float(delay_delta),      # days late (real - scheduled)
    float(total_quantity),   # total units ordered
    float(total_sales),      # total monetary value
    float(profit_margin),    # profit / sales ratio
    float(discount_ratio),   # average discount across lines
]], columns=model_data["features"])

# Normalize using the same scaler used during training
features_scaled = scaler.transform(features_df)

# Predict: 0 = normal, 1 = fraud/unusual
is_knn_fraud = int(knn.predict(features_scaled)[0])

# Final status assignment:
if is_knn_fraud == 1 or doc.get("status") == "SUSPECTED_FRAUD":
    anomaly_status = "unusual"
elif delay_delta > 3:
    anomaly_status = "delay anomaly"
else:
    anomaly_status = "valid"
```

The .pkl Model File

KNN model structure

```
# knn_anomaly_model.pkl contains a dict:
{
  "knn":      KNeighborsClassifier (trained model),
  "scaler":   StandardScaler (to normalize features identically to training),
  "features": ["delay_delta", "total_quantity", "total_sales",
               "profit_margin", "discount_ratio"]
}

# Loaded once at startup:
with open("knn_anomaly_model.pkl", "rb") as f:
    KNN_MODEL_DATA = pickle.load(f)
```

REBUILD WARNING: The model path is hardcoded to C:\Users\Sabor\Desktop\... — a Windows absolute path. For any rebuild or deployment, change this to `os.path.join(os.path.dirname(__file__), '../processed_data/knn_anomaly_model.pkl')` or use an environment variable.

Prophet Demand Forecasting

Meta's Prophet library predicts future daily demand by learning trends and seasonal patterns from historical sales data. The full pipeline runs as a background task.

Full Retraining Pipeline

services/forecast_service.py — retraining

```
async def retrain_demand_forecast(db, product_id):
    # 1. Fetch historical sales from MongoDB (last 180 days max)
    cursor = db["forecasts"].find({"sales": {"$ne": None}, "product_id": product_id})
    records = [doc async for doc in cursor]

    # 2. Build DataFrame
    df = pd.DataFrame([{"ds": pd.to_datetime(r["date"]), "y": float(r["sales"])}
                       for r in records])
    df = df.groupby("ds").agg({"y": "sum"}).reset_index()

    # 3. Log-transform to handle skewed distributions
    df["y"] = np.log1p(df["y"]) # log(1 + y)

    # 4. Fit Prophet in a thread (avoids blocking the async event loop)
    model = Prophet(yearly_seasonality=False, weekly_seasonality=True,
                    uncertainty_samples=0) # 0 = 5-10x faster
    await asyncio.to_thread(model.fit, df[['ds', 'y']])

    # 5. Predict next 90 days
    future = model.make_future_dataframe(periods=90, include_history=False)
    forecast = model.predict(future)

    # 6. Inverse transform: e^yhat - 1, clipped to 0
    forecast['yhat'] = np.expm1(forecast['yhat']).clip(lower=0.0)

    # 7. Save future forecasts to MongoDB (replace previous ones)
    await db["forecasts"].delete_many({"sales": None, "product_id": product_id})
    await db["forecasts"].insert_many([...new 90-day documents...])
```

Performance Optimisations Used

Optimisation	Effect
uncertainty_samples=0	5-10x speedup — skips confidence interval computation
Last 180 days only	Caps training data to prevent slow training on large datasets
asyncio.to_thread(model.fit)	Runs CPU-heavy fit() in thread pool, does not block the async loop
include_history=False	make_future_dataframe() only generates future dates, not history
log1p / expm1 transform	Compresses skewed sales data for better model fit, inverts after predict

Local LLM via Ollama

Both AI summary endpoints — the supply chain overview and the per-product forecast explanation — call a locally running Ollama instance. No external API, no cost, fully private.

How Ollama is Called

Ollama API call pattern

```
async with httpx.AsyncClient(timeout=60.0) as client:
    # 1. Discover which models are installed locally
    models_resp = await client.get("http://localhost:11434/api/tags")
    installed = [m["name"] for m in models_resp.json().get("models", [])]

    # 2. Pick the best available model (preference order)
    preferred = ["qwen2.5:7b", "qwen2.5:latest", "llama3.1", "mistral"]
    model_to_use = next(
        (m for pref in preferred for m in installed if m.startswith(pref)),
        installed[0] if installed else "qwen2.5:7b"
    )

    # 3. Generate text (stream=False = wait for full response)
    response = await client.post("http://localhost:11434/api/generate", json={
        "model": model_to_use,
        "prompt": prompt,
        "stream": False
    })
    explanation = response.json().get("response", "").strip()
```

Prompt Engineering Pattern

Prompt structure in kpis.py

```
prompt = (
    f"You are a supply chain AI analyst. Write a concise 2-3 sentence "
    f"executive summary of the current supply chain status.\n"
    f"- Total sales orders analyzed: {total_orders}.\n"
    f"- KNN unusual fraud transactions flagged: {unusual_count}.\n"
    f"- Critical shipping delays detected: {len(delayed)}.\n\n"
    f"Write a professional summary. State numbers explicitly. "
    f"Do NOT use bullet points, markdown, or greetings." # <- constraint
)
```

Graceful Fallback

If Ollama is not running (no GPU, production server), the code catches the connection error and returns a rule-based template. The Angular frontend always receives a response — it never knows Ollama failed.

Graceful fallback pattern

```
try:
    explanation = await call_ollama(prompt)
    if explanation:
        return {"explanation": explanation}
except Exception:
    pass # Ollama not running? No crash, no 500 error.

# Rule-based fallback (always works):
return {"explanation":
        f"KNN classifier flagged {unusual_count} unusual transactions. "
        f"Delayed orders: {delayed_orders_str}. Recommend immediate review."
}
```

Complete HTTP + Token Flow

What actually happens, byte by byte, from the user clicking Login to seeing orders on screen.

Step 1 — Login

Full login sequence

Angular sends:

```
POST http://127.0.0.1:8000/api/auth/login
Content-Type: application/json
Body: { "email": "admin@example.com", "password": "secret" }
```

FastAPI responds:

```
{ "access_token": "eyJhbGciOi...", "token_type": "bearer" }
```

Angular stores:

```
localStorage.setItem('access_token', 'eyJhbGciOi...')
```

Angular then calls `fetchCurrentUser()` with the new token:

```
GET /api/auth/me
Authorization: Bearer eyJhbGciOi...
```

FastAPI responds:

```
{ "id": "abc123", "email": "...", "role": "admin", ... }
```

BehaviorSubject emits the user object -> guards allow navigation -> redirect to /

Step 2 — Every Protected Request

Protected request lifecycle

Angular reads token and builds headers:

```
const headers = new HttpHeaders({
  'Authorization': `Bearer ${localStorage.getItem('access_token')}`
});
```

Angular sends:

```
GET http://127.0.0.1:8000/api/kpis/orders
Authorization: Bearer eyJhbGciOi...
```

FastAPI's `get_current_admin()` Dependency runs FIRST:

1. `HTTPBearer()` extracts "Bearer eyJhbGciOi..." from the header
2. `jwt.decode()` verifies signature + checks expiry
3. Extracts "sub" = admin's MongoDB `_id`
4. Fetches admin document from MongoDB
5. Injects admin dict into the route handler

Route handler runs normally and returns data.

Step 3 — Token Expiry

Token expiry flow

Token expires after 120 minutes (config.py: ACCESS_TOKEN_EXPIRE_MINUTES = 120)

FastAPI responds with:

```
401 Unauthorized
{ "detail": "Could not validate credentials" }
```

Angular's catchError in fetchCurrentUser():

- > calls this.logout()
- > logout() removes token from localStorage
- > userStateSubject.next(null)
- > authGuard sees null -> redirects to /login
- > User sees the login page again

How to Rebuild from Scratch

If you had to rebuild this project from zero, here is the exact sequence — the right order matters because each phase depends on the previous one.

Phase 1 — Backend Foundation

- `pip install fastapi uvicorn motor pydantic-settings python-jose passlib[bcrypt] python-multipart httpx`
- Create `core/config.py` with `pydantic-settings`: `JWT_SECRET_KEY`, `MONGODB_URI`, `DATABASE_NAME`, `ACCESS_TOKEN_EXPIRE_MINUTES`
- Create `core/database.py`: `AsyncIOMotorClient` singleton, `connect_to_database()`, `get_db()` dependency function
- Create `core/security.py`: `get_password_hash()`, `verify_password()` with `passlib`, `create_access_token()` with `python-jose`
- Create `models/user.py`: `AdminBase`, `AdminCreate`, `AdminOut` (no password field!), `LoginRequest` with `EmailStr`
- Create `services/auth_service.py`: `get_current_admin()` that decodes JWT and looks up admin in MongoDB
- Create `routers/auth.py`: `POST /login` (verify password, return token) and `GET /me` (return current admin)
- Create `main.py`: FastAPI app with lifespan hook, `CORSMiddleware` for `localhost:4200`, `include_router` x3 with `/api` prefix
- Test with Swagger UI: run `"uvicorn app.main:app --reload"` and open `http://localhost:8000/docs`

Phase 2 — Angular Foundation

- `ng new front-end --standalone --routing` (Angular 21+ uses standalone by default)
- In `app.config.ts`: add `provideRouter(routes)` and `provideHttpClient()` to `providers` array
- Create `services/auth.ts`: `BehaviorSubject(undefined)`, `login()`, `logout()`, `fetchCurrentUser()` called in constructor
- Create `auth.guard.ts`: `filter(user !== undefined)`, `take(1)`, `map(user => user ? true : router.navigate(/login))`
- Create `login.guard.ts`: same filter pattern, redirect logged-in users with role check (`supplier -> /supplier`, else `-> /`)
- Set up `app.routes.ts`: `/login` with `canActivate:[loginGuard]`, parent `""` with `canActivate:[authGuard]` and children
- Create Login component: `[(ngModel)]` on email/password, inject `Auth`, call `auth.login().subscribe()`, handle errors with `signal`
- Test: `ng serve`, navigate to `http://localhost:4200` — should redirect to `/login`. Login should redirect to `/`

Phase 3 — Feature Pages

- Add `kpis.py` router with GET `/orders`: fetch `sales_orders`, join with `products`, compute `total_sales/profit_margin/delay_delta`
- Add GET `/purchases`, GET `/products`, GET `/forecasts` with optional `product_id` query param
- Add GET `/overview/explain`: compile stats, call Ollama, fallback to template string
- Create `SalesOrder` component: inject `HttpClient` + `Auth`, `loadDashboardData()` with 3 parallel HTTP calls, computed getters for filtered/paginated orders
- Add scatter plot: `buildScatterPlot()` maps each order to `(cx, cy)` coordinates using min-max normalisation
- Add calendar: `buildCalendarGrid()` calculates `firstDayIndex`, fills 35/42 cells, maps events to `dateStr`
- Create `DemandForecast` component: load `Id` pattern, safety timer, 5s polling, dynamic `stddev` thresholds for `stockout`

Phase 4 — ML Features

- KNN Model: train in Jupyter (`KNeighborsClassifier` + `StandardScaler`), export with `pickle.dump({"knn": model, "scaler": scaler, "features": names})`
- Fix the hardcoded Windows model path: use `os.path.join(os.path.dirname(__file__), "../processed_data/knn_anomaly_model.pkl")`
- Create `services/forecast_service.py`: `retrain_demand_forecast()` with `log1p`, `asyncio.to_thread(model.fit)`, `expm1` inverse, insert 90-day forecasts
- Add `BackgroundTasks` to GET `/forecasts`: if no December forecasts found, kick off retraining without blocking response
- Install Ollama: `brew install ollama` (Mac) or see ollama.com. Run `"ollama pull qwen2.5:7b"`
- Add `generate_forecast_explanation()` in `forecast_service.py`: build prompt with real numbers, call `localhost:11434/api/generate`, template fallback

Things to Fix in a Rebuild

1. Hardcoded KNN model path to `C:\Users\Sabor\...` — use a relative path or env variable. 2. No Angular HTTP interceptor — every component manually adds the Bearer token header. Add an interceptor to do it automatically. 3. API base URL is hardcoded as `'http://127.0.0.1:8000/api'` in multiple files — move it to `environment.ts`. 4. JWT secret is hardcoded in `config.py` — should only come from a `.env` file, never committed to git. 5. Placeholder components (`chatbot`, `insight`, `admin-dashboard`, `supplier`) are completely empty.